Candidate-Fate Accounting for Transparent Sensor Diagnostic Pipeline Search

Haotao Xie
Hangzhou International Innovation Institute, Beihang University
Hangzhou, China
haotaoxie@buaa.edu.cn

Yutian Chen
Hangzhou International Innovation Institute, Beihang University
Hangzhou, China
25601153@buaa.edu.cn

Yangqi Liu
College of Cyber Security, Jinan University
Guangzhou, China
yangqilau@163.com

Xiaoyu Jiang✉
Hangzhou International Innovation Institute, Beihang University
Hangzhou, China
jiangxiaoyu@buaa.edu.cn

Abstract

Industrial sensor diagnostics relies on preprocessing, representation, and classification pipelines, making automated pipeline search useful for reducing manual design cost. However, existing automated machine/deep learning (AutoML/AutoDL) reports typically retain only fitted trials, scores, and winners, omitting generated candidates that are invalid, pruned, skipped, cached, or unfitted. This omission limits reviewers' ability to check signal constraints, budget use, and unevaluated legal alternatives. To address this, we propose candidate-fate accounting, a candidate-level audit framework for diagnostic search traces. It records each observed candidate as auditable evidence: hashes merge repeated observations, legality checks flag invalid candidates, allocation rationales explain budget decisions, and a closed fate ledger assigns one terminal fate to each candidate. Experiments on three bearing-diagnostic datasets show that the framework detects invalid candidates and identifies 30–41 candidates omitted by fitted-trial-only reports, with closed fate records verifying complete candidate accounting while maintaining competitive diagnostic performance. The code is available at https://github.com/XXIE999/candidate-fate-accounting.

CCS Concepts

• **Computing methodologies** → **Machine learning**; • **Information systems** → *Data mining*; • **Applied computing** → Industry and manufacturing.

Keywords

industrial sensor diagnostics, search transparency, candidate-fate accounting

ACM Reference Format:
Haotao Xie, Yutian Chen, Yangqi Liu, and Xiaoyu Jiang. 2026. Candidate-Fate Accounting for Transparent Sensor Diagnostic Pipeline Search. In *Proceedings of the 35th ACM International Conference on Information and Knowledge Management (CIKM '26), November 7–11, 2026, Rome, Italy.* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3799682.3839938

ACM ISBN 979-8-4007-2539-5/2026/11
https://doi.org/10.1145/3799682.3839938

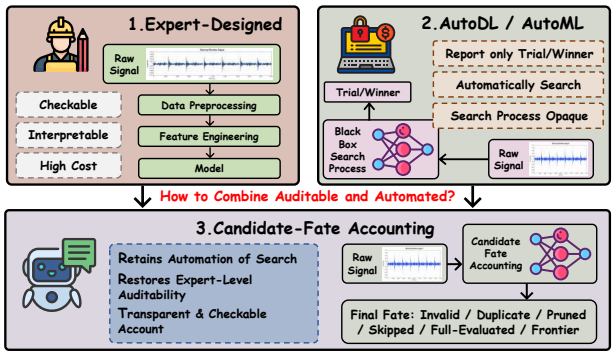

Figure 1: Motivating comparison among expert-designed workflows, AutoML/AutoDL search, and candidate-fate accounting.

1 Introduction

Industrial sensor diagnostics depends on pipelines that convert machine signals into reliable fault decisions [31]. In deployed maintenance settings, these pipelines must be both accurate and inspectable: engineers need to know which signal transforms are admissible, which classifiers can consume each representation, and why a selected model is plausible [14, 25]. Expert-built pipelines provide this reviewability, but the same reasoning is costly to repeat across machines, channels, and operating regimes [32]. Automated pipeline search [5, 20, 24] reduces this manual burden by generating and evaluating candidate preprocessing, representation, and classification pipelines. However, automation should not remove the evidence needed to review the search process. A useful diagnostic search report should show not only which pipeline won,

but also what happened to generated candidates that were never fitted. Figure 1 summarizes this contrast.

Existing work [13, 19, 33] addresses parts of this problem but does not close the generated-candidate audit gap. AutoML and AutoDL methods report fitted configurations; hyperparameter-optimization frameworks such as Optuna record scheduled-trial states such as completed, failed, or pruned trials [1, 7, 12, 17, 26]; grammar and validity methods reject illegal programs; and provenance systems track executed artifacts [2, 8–10, 18, 21, 22, 27, 29]. These tools are useful, but their reporting units are usually fitted configurations, scheduled trials, rejected programs, or executed artifacts rather than canonical generated candidates. As a result, repeated proposals, type-invalid candidates that never become trials, and legal candidates skipped by budget or cache decisions may lack one terminal explanation, leaving search validity and budget use difficult to audit at the candidate level. Candidate-fate accounting complements rather than replaces these systems: it adds a canonical candidate-level partition that also covers invalid and non-executed alternatives. This common reporting unit supports reproducible, optimizer-independent comparison of legality and budget traces across AutoML search policies.

A simple diagnostic trace illustrates the gap. Suppose a search generates a short-time Fourier transform (STFT) map with logistic regression, a z-score plus log-mel support vector machine (SVM) pipeline, and a highpass raw-vector XGBoost pipeline, but fits only the third candidate. A fitted-trial report records only the evaluated pipeline and score, hiding that the first is type-invalid and the second is legal but cost-blocked. These hidden outcomes matter under small budgets because they expose legality checks, budget allocation, cache reuse, and untested legal alternatives.

To address this gap, we design candidate-fate accounting, a candidate-level audit framework for emitted diagnostic search traces. The framework closes the record over observed canonical candidates rather than enumerating candidates that never appear. It uses stable hashes to merge repeated observations, typed legality checks to expose type and semantic failures before fitting, allocation rationales to explain budget decisions for legal candidates, and a closed fate ledger with Δ_{close} to verify that each observed candidate receives exactly one terminal fate. Thus, non-fitted candidates become reportable evidence rather than optimizer bookkeeping. Allocation policies remain replaceable: guided ledger search is one allocator designed in our experiments, while candidate-fate accounting is the paper's main contribution. Figure 2 summarizes the workflow.

We evaluate candidate-fate accounting on three bearing-diagnostic datasets: the Case Western Reserve University (CWRU) bearing dataset, the University of Ottawa bearing dataset (Ottawa), and the Southeast University (SEU) bearing dataset. The experiments test whether the framework detects invalid candidates before fitting, accounts for candidates omitted by fitted-trial-only reports with complete fate accounting, and preserves useful diagnostic performance under a controlled protocol.

The main contributions are:

(1) We define the generated-candidate audit gap in automated industrial diagnostic search, showing why non-fitted candidates should be treated as evidence about validity, budget use, and untested legal alternatives.

(2) We design a typed candidate-fate accounting framework that maps observed canonical candidates to auditable legality, allocation rationale, terminal fate, and closure evidence.

(3) We evaluate the framework on CWRU, Ottawa, and SEU through invalid-candidate probes, closed fate ledgers, allocation checks, and controlled-protocol diagnostic performance.

2 Method

We formalize candidate-fate accounting as a reporting contract over emitted diagnostic search traces. As shown in Figure 2, the framework records typed legality evidence, allocation rationale, and terminal fates in closed fate ledger. These records make signal constraints, budget use, and unevaluated legal alternatives inspectable while keeping the audit layer optimizer-independent. Section 2.1 defines legal diagnostic candidates, Section 2.2 defines budget allocation, and Section 2.3 assigns final fates.

2.1 Typed Legality Evidence

A generated candidate is a diagnostic program $c = (p, r, e)$, where p is a preprocessing chain, r is a signal representation, and e is an estimator. The objects exchanged by these components have declared types, such as waveform, vector, time–frequency map, and patch sequence. Each primitive u declares an input type $\tau_{\text{in}}(u)$, an output type $\tau_{\text{out}}(u)$, and lightweight metadata for semantic checks. The first legality condition is type compatibility:

$$\mathcal{C}_{\text{type}} = \{c = (p, r, e) \,:\, \tau_{\text{out}}(p) = \tau_{\text{in}}(r),\ \tau_{\text{out}}(r) = \tau_{\text{in}}(e)\}.$$

The set $\mathcal{C}_{\text{type}}$ contains candidates whose adjacent signatures agree. It captures syntactic feasibility before any search policy is considered. This rule turns signal-type constraints into pre-fit evidence by rejecting errors such as feeding a map representation to a vector-only classifier. A deterministic semantic map $\rho : \mathcal{C}_{\text{type}} \to \{0, 1\}$ resolves conflicts by conservative rejection: a missing-value primitive is invalid when dataset metadata reports zero missingness, while a second z-score normalization or filtering step is invalid as redundant. Fixed rule precedence records one reason when checks overlap.

The second step distinguishes intrinsic legality from budget admissibility. Given budget B and its cost guard $\text{CostOK}(c; B)$, we define

$$\text{Legal}(c) \equiv c \in \mathcal{C}_{\text{type}} \wedge \rho(c) = 1,$$
$$\text{Admissible}(c; B) \equiv \text{Legal}(c) \wedge \text{CostOK}(c; B).$$

Thus $\mathcal{C}_{\text{leg}} = \{c \,:\, \text{Legal}(c)\}$ is the legal candidate space independent of budget. Type/semantic-incompatible candidates are invalid before fitting, while legal candidates blocked by cost, cache, or termination remain legal and later receive a non-evaluated fate.

2.2 Allocation Rationale

Allocation rationale makes budget use reviewable without changing candidate legality. At search step t, fitting budget [16] may be spent only on candidates that are generated, legal, not cached, and not already assigned a terminal fate. Let $\mathcal{G}_t$ be generated candidates, $\mathcal{H}_{t-1}$ be cached canonical hashes, $h(c)$ be the stable hash of candidate c, and $\ell(c)$ be its current ledger state. The allocatable set

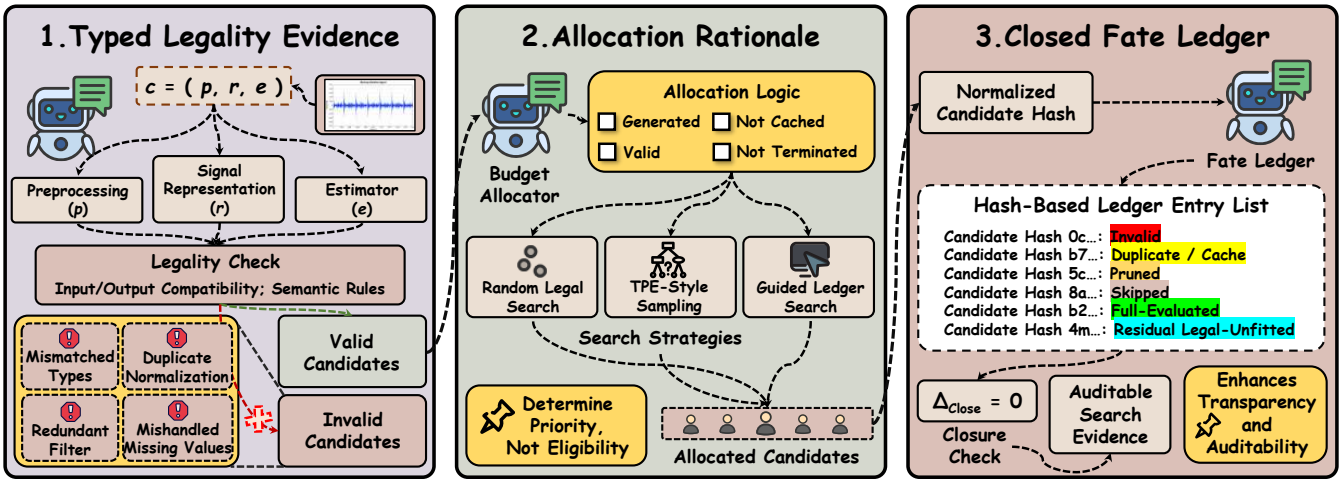

Figure 2: Overview of candidate-fate accounting. Typed legality checks classify generated candidates; a replaceable allocator prioritizes eligible ones; and a hash-based ledger consolidates repeats and assigns one terminal fate—including non-fitted outcomes—to each observed candidate. The closure condition $\Delta_{\text{close}} = 0$ verifies complete, non-overlapping accounting.

is

$$\mathcal{A}_t = \{c \in \mathcal{G}_t \cap \mathcal{C}_{\text{leg}} : h(c) \notin \mathcal{H}_{t-1},\ \ell(c) \text{ unassigned}\}.$$

Search policies operate only on $\mathcal{A}_t$: RLS [4] samples uniformly, the TPE-style sampler [3] adapts primitive distributions from observed validation macro-F1, and GLS [15] uses MCTS-style selection over grammar-approved macro actions. They change only legal-candidate priority, not legality, budget eligibility, cache handling, or terminal fate.

Optional training-split, label-free descriptors such as impulsiveness, spectral entropy, support, and missingness define r_{guide} by adjusting primitive-family priority, but cannot bypass legality or ledger rules.

2.3 Closed Fate Ledger

The closed ledger reports the fate of every observed canonical candidate, including candidates that were generated but never fitted. The reporting unit is a canonical hash because one diagnostic program may appear through proposal, cache, probe, or evaluation records. In our implementation, $h(c)$ is a SHA-256-derived identifier over a deterministic sorted-key serialization of the ordered preprocessing steps, representation, estimator, and their parameters. Each hash bucket stores the serialized candidate, legality result, allocation reason, and observed event types. Repeated proposal, cache, probe, or evaluation records update the existing bucket rather than increasing U. Because component order and parameters are included, the same configuration maps to one identity independently of its event path, while structurally different pipelines remain distinct. Ledger construction therefore separates identity resolution from fate assignment: records are first consolidated by $h(c)$, precedence is then applied per bucket, and closure is computed over U unique entries. This prevents repeated observations from inflating generated-candidate counts while preserving the event evidence needed to justify the final state.

Let $\bar{\mathcal{G}}_T$ be the observed canonical generated set at the end of a run. The fate function $\ell : \bar{\mathcal{G}}_T \to \mathcal{S}$ maps each candidate to one state in $\mathcal{S} = \{s_{\text{inv}}, s_{\text{dup}}, s_{\text{prn}}, s_{\text{skp}}, s_{\text{eval}}, s_{\text{res}}\}$: invalid, duplicate/cache, pruned, skipped, full-evaluated, or residual legal-unfitted.

Fates are assigned by a fixed precedence order so that each observed canonical candidate contributes to one table cell. The order makes late evidence decisive when a candidate is first proposed cheaply and later fitted. Candidates failing type or semantic checks become invalid. Any candidate that consumes fitting budget becomes full-evaluated, even if earlier records only proposed or probed it. Legal candidates observed only through cache reuse become duplicate/cache. Legal non-duplicates removed by cost, complexity, or multi-fidelity guards become pruned. Legal candidates selected for execution but blocked before fitting become skipped. Remaining legal non-duplicates that are observed but never fitted become residual legal-unfitted.

The closure check tests whether terminal fates form a mutually exclusive and exhaustive partition of the observed set:

$$\bar{\mathcal{G}}_T = \dot{\bigcup}_{s \in \mathcal{S}} \bar{\mathcal{G}}_T^s, \qquad \Delta_{\text{close}} = |\bar{\mathcal{G}}_T| - \sum_{s \in \mathcal{S}} |\bar{\mathcal{G}}_T^s| = 0.$$

Here $\dot{\bigcup}$ denotes a disjoint union, $\bar{\mathcal{G}}_T^s$ is the subset assigned fate s, and Δ_{close} is zero only when no observed canonical candidate is missing or double-counted.

For each observed canonical candidate, the evidence tuple is

$$\mathcal{E}(c) = (r_{\text{type}}(c), r_{\text{guide}}(c), \ell(c)),$$

where r_{type} is the legality rationale, r_{guide} is optional allocation metadata or rationale, and $\ell(c)$ is the terminal fate. The run-level report is

$$\mathcal{R}_T = (c^\star, \{\mathcal{E}(c) : c \in \bar{\mathcal{G}}_T\}, B_T, \Delta_{\text{close}}).$$

Here $c^\star$ is the selected pipeline and $B_T = |\bar{\mathcal{G}}_T^{s_{\text{eval}}}| \le B$. This report states which pipeline won, why candidates were ruled out, where budget was spent, and which legal alternatives remained

Table 1: Closed $B = 50$ GLS ledger counts under typed legal search, averaged over three seeds.

Dataset	Unique Generated	Invalid	Pruned	Skipped	Duplicate/ Cache	Full Evaluated	Residual Legal-Unfitted	Closure Gap
CWRU	49.7±0.5	0±0	6.7±0.9	23.0±2.8	0±0	20.0±2.9	0±0	0
Ottawa	49.7±0.5	0±0	20.3±2.9	18.7±4.7	2.3±0.9	8.3±2.1	0±0	0
SEU	49.7±0.5	0±0	6.3±2.1	23.7±3.1	4.3±2.1	15.3±2.5	0±0	0

unseen by fitting. For N trace records, U unique candidates, and serialized candidate size L, ledger construction costs $O(NL)$ hashing plus expected $O(N)$ hash-table updates; the in-memory unique-candidate ledger needs $O(U)$ storage, and closure is $O(U)$, with no additional model fitting. Each incoming record requires one hash and an expected $O(1)$ lookup/update. In our $B = 50$ GLS runs, the main JSON ledger occupies 32.8–37.7 KB per run; exact disk use depends on schema and serialization. At the accounting layer, a new domain supplies a canonical serializer, type/semantic rules, and a mapping from search events to the six terminal fates; hash consolidation, precedence, and closure remain unchanged.

3 Experiments

We evaluate four research questions (RQs) aligned with the audit framework: whether typed legality exposes invalid candidates before fitting (RQ1), whether the candidate-fate ledger closes over observed generated candidates (RQ2), whether allocation behavior is comparable under the same legal space (RQ3), and whether audited search still returns useful diagnostic pipelines under a controlled protocol (RQ4).

3.1 Experimental Setup

Protocol. We use a controlled small-budget protocol to compare audit counts and diagnostic utility across search policies. For each dataset and seed, all main search conditions share the same typed legal space, primitive cost model, split, and macro-F1 metric, with at most $B = 50$ full model-fitting attempts. This cap applies to fitted candidates, not generated candidates: generated candidates may instead be skipped, pruned, cached, or recorded as residual legal-unfitted. Results are averaged over seeds 42, 43, and 44.

Datasets. We evaluate three bearing-diagnostic datasets with dataset-appropriate window-level splits. CWRU [11, 23, 28] and Ottawa use approximately 60/20/20 train/validation/test splits. SEU uses a cross-condition split 30_2 → 20_0, with the source condition for training and the target condition split equally for validation and testing.

3.2 Typed Legality Probe

RQ1 tests whether typed legality exposes invalid candidates before fitting. We generate 100 weakly constrained skeletons over the shared primitive inventory and label each skeleton with type and semantic checks.

The probe yields 48 type-invalid, 20 semantic-invalid, and 32 legal skeletons. Type failures capture object mismatches such as time–frequency maps followed by vector-only classifiers; semantic failures capture conservative metadata conflicts such as duplicate normalization or repeated filtering. The 68/100 invalid count is a legality-layer stress test, not an estimate of typed main-run invalidity: because the main searches operate within the typed legal space, zero invalid candidates there is expected. Rather than weakening the audit claim, this zero-invalid main-run outcome makes the invariant checkable: the ledger confirms that invalid candidates do not consume fitting budget, and the weak probe identifies the failures that less constrained generation would need to catch and explain.

3.3 Closed Ledger Accounting

RQ2 tests whether candidate-fate accounting accounts for emitted candidates dropped by fitted-trial reporting and verifies complete accounting over observed canonical candidates. Each record stores legality evidence, allocation rationale when available, a terminal fate, and a reason. Table 1 gives a $B = 50$ GLS accounting example aggregated over three seeds.

The main finding is that fitted-trial-only reporting omits substantial trace evidence: the ledger records about 30, 41, and 34 non-full-evaluated canonical candidates on CWRU, Ottawa, and SEU, respectively. The closure check verifies that these fate counts form a complete and non-overlapping partition, with all GLS ledger rows in Table 1 satisfying $\Delta_{\text{close}} = 0$.

For example, fitted-trial-only reporting on Ottawa exposes only 8.3±2.1 full-evaluated candidates, whereas the ledger attributes 41.3 others on average to pruning, skipping, or duplicate/cache. The latter comprise 20.3 pruned, 18.7 skipped, and 2.3 duplicate/cache candidates, so the shortfall from generated candidates to fitted trials becomes attributable rather than unexplained. The interpretability is process-level: legality decisions, candidate fate, and budget consequence, not post-hoc explanations of a fitted model. These fates make the search reviewable but do not rank optimizers.

3.4 Early Allocation

RQ3 uses GLS as an allocator case study and compares allocation behavior with the legal space and budget fixed. All rows share typed legality, cache rules, semantic guards, and fitting budget. Best F1 $\leq$ 30 is the best validation macro-F1 within 30 full evaluations; Target Success counts seeds that reach the final RLS validation score, used as a dataset-specific reference target; Evaluations to Target measures the first full-evaluation index at which a run reaches the final RLS validation score, computed only over successful seeds. Low Target Success therefore indicates unstable early allocation.

Table 2: Early allocation under a shared typed legal space. Bold marks the best F1 and, among 3/3-success rows, the fewest Evaluations to Target.

Dataset	Method	Best F1≤30 ↑	Target Success	Evaluations to Target ↓
CWRU	RLS	0.9942±0.0009	3/3	26.00±7.87
CWRU	TPE	**0.9982**±0.0009	3/3	19.00±4.32
CWRU	GLS	**0.9982**±0.0002	3/3	**7.00±4.55**
Ottawa	RLS	0.9673±0.0063	3/3	9.00±5.89
Ottawa	TPE	0.9639±0.0038	1/3	3.00±0.00
Ottawa	GLS	**0.9856±0.0026**	3/3	**3.67±1.70**
SEU	RLS	0.6751±0.0458	3/3	28.00±3.56
SEU	TPE	0.7090±0.0167	1/3	27.00±0.00
SEU	GLS	**0.7646±0.0185**	3/3	**7.33±6.85**

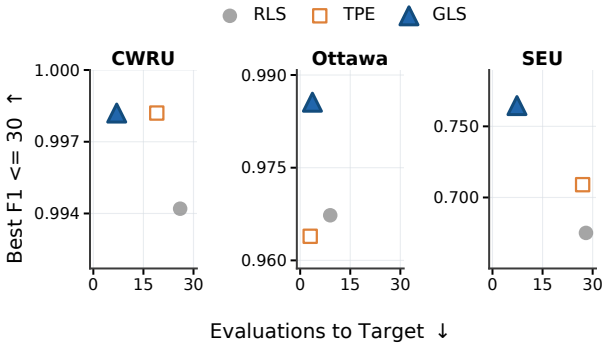

Figure 3: Best validation macro-F1 within 30 full evaluations versus Evaluations to Target; upper-left is better.

Table 2 and Figure 3 summarize early-allocation speed and budget-30 utility. GLS gives the most consistent early-allocation profile: it reaches the random-search target in 3/3 seeds on all datasets, requires the fewest Evaluations to Target among 3/3-success rows, attains the best budget-30 F1 on Ottawa and SEU, and ties TPE to four decimals on CWRU. This should be read as an auditability check, not as a broad optimizer ranking: allocation choices can be reported alongside fitted-trial outcomes under the same legal space.

3.5 Protocol Utility

RQ4 asks whether the audit framework retains controlled-protocol diagnostic utility. Table 3 reports controlled-protocol final-test macro-F1 for selected same-space search pipelines, fixed diagnostic recipes, a one-dimensional convolutional neural network (1D-CNN), and AutoML references. Random forest (RF) denotes the estimator used in two fixed recipes. Search rows are matched by split, channel, windowing, label map, metric, and data limit; neural and AutoML rows use different input representations and serve only as scale references.

Table 3: Protocol-scoped final-test macro-F1; search rows show three-seed mean (standard deviation), with the best matched search policy per dataset in bold.

Method	CWRU F1 ↑	Ottawa F1 ↑	SEU F1 ↑
RLS	0.9951(0.0004)	0.9851(0.0046)	0.7130(0.0514)
TPE	**0.9978(0.0028)**	0.9691(0.0119)	0.7128(0.0242)
GLS	0.9968(0.0000)	**0.9862(0.0032)**	**0.7341(0.0134)**
Statistical + RF	0.8200	0.6216	0.0748
Spectral classical	0.7068	0.6181	0.0593
Envelope classical	0.5364	0.5926	0.5899
Wavelet + RF	0.8324	0.7490	0.0901
1D-CNN	0.7414	0.0667	0.2380
AutoGluon [6]	0.9172	0.8776	0.2647
FLAML [30]	0.9022	0.8821	0.1371

Table 3 shows that accountable search retains controlled-protocol diagnostic utility. Same-space search rows achieve high macro-F1 on CWRU and Ottawa; among matched search policies, TPE is highest on CWRU while GLS is highest on Ottawa and SEU. Because selected pipelines vary across settings, the report records the selected family, transform, estimator, and seed. These results support the scoped claim that candidate-fate accounting can expose terminal fates and allocation outcomes while still selecting plausible final models.

4 Conclusion

Candidate-fate accounting addresses the generated-candidate audit gap in automated diagnostic search. By merging repeated trace records, recording legality and allocation rationales, and checking ledger closure with Δ_{close}, it turns rejected, skipped, cached, and unfitted candidates into reviewable evidence. Experiments on CWRU, Ottawa, and SEU show that the framework exposes invalid skeletons, attributes non-full-evaluated candidates to explicit fates, and retains useful final-test performance under a controlled protocol. The accounting contract can be adapted through domain-specific candidate schemas and semantic rules, but empirical cross-domain validation remains future work.

Acknowledgments

This work was supported in part by the National Natural Science Foundation of China under Grant 62403425, in part by the Zhejiang Provincial Natural Science Foundation of China under Grant LMS26F030019, in part by the Hangzhou Natural Science Foundation under Grant 2025SZRJJ2330, in part by the Youth Talent Support Project of the Zhejiang Provincial Association for Science and Technology, and in part by the Jiangsu Provincial Scientific Research Center of Applied Mathematics under Grant BK20233002.

Generative AI (GenAI) Usage Disclosure

The authors used generative AI tools in a limited assistive capacity for manuscript language polishing and for code debugging/checking. These tools were not used to generate experimental data, alter results, or make scientific decisions; all code, results, technical claims, and final manuscript text were reviewed and verified by the authors.

References

[1] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. 2019. Optuna: A Next-generation Hyperparameter Optimization Framework. *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* (2019). https://api.semanticscholar.org/CorpusID:196194314

[2] Khalid Belhajjame, Reza B'Far, James Cheney, Sam Coppens, Stephen Cresswell, Yolanda Gil, Paul Groth, Graham Klyne, Timothy Lebo, Jamie McCusker, Simon Miles, James D. Myers, Satya Sanket Sahoo, and Curt Tilmes. 2013. PROV-DM: The PROV Data Model. https://api.semanticscholar.org/CorpusID:65235238

[3] James Bergstra, Rémi Bardenet, Yoshua Bengio, and Balázs Kégl. 2011. Algorithms for Hyper-Parameter Optimization. In *Neural Information Processing Systems*. https://api.semanticscholar.org/CorpusID:11688126

[4] James Bergstra and Yoshua Bengio. 2012. Random search for hyper-parameter optimization. *Journal of machine learning research* 13, 2 (2012).

[5] Iddo Drori, Yamuna Krishnamurthy, Rémi Rampin, Raoni Lourenço, Jorge Piazentin Ono, Kyunghyun Cho, Cláudio T. Silva, and Juliana Freire. 2021. AlphaD3M: Machine Learning Pipeline Synthesis. *ArXiv* abs/2111.02508 (2021). https://api.semanticscholar.org/CorpusID:198940685

[6] Nick Erickson, Jonas W. Mueller, Alexander Shirkov, Hang Zhang, Pedro Larroy, Mu Li, and Alex Smola. 2020. AutoGluon-Tabular: Robust and Accurate AutoML for Structured Data. *ArXiv* abs/2003.06505 (2020). https://api.semanticscholar.org/CorpusID:212725762

[7] Stefan Falkner, Aaron Klein, and Frank Hutter. 2018. BOHB: Robust and Efficient Hyperparameter Optimization at Scale. *ArXiv* abs/1807.01774 (2018). https://api.semanticscholar.org/CorpusID:49571505

[8] Luís Ferreira, André Pilastri, Filipe Romano, and Paulo Cortez. 2022. Using supervised and one-class automated machine learning for predictive maintenance. *Applied Soft Computing* 131 (2022), 109820. doi:10.1016/j.asoc.2022.109820

[9] Matthias Feurer, Aaron Klein, Katharina Eggensperger, Jost Springenberg, Manuel Blum, and Frank Hutter. 2015. Efficient and Robust Automated Machine Learning. In *Advances in Neural Information Processing Systems*, C. Cortes, N. Lawrence, D. Lee, M. Sugiyama, and R. Garnett (Eds.), Vol. 28. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2015/file/11d0e6287202fced83f79975ec59a3a6-Paper.pdf

[10] Russul H. Hadi, Haider Najy Hady, Ahmed Mudheher Hasan, Ammar Abdulhussein Lafta Al-Jodah, and Amjad Jaleel Humaidi. 2023. Improved Fault Classification for Predictive Maintenance in Industrial IoT Based on AutoML: A Case Study of Ball-Bearing Faults. *Processes* (2023). https://api.semanticscholar.org/CorpusID:258749232

[11] Jacob Hendriks, Patrick Dumond, and David Knox. 2022. Towards better benchmarking using the CWRU bearing fault dataset. *Mechanical Systems and Signal Processing* (2022). https://api.semanticscholar.org/CorpusID:245603873

[12] Frank Hutter, Holger H. Hoos, and Kevin Leyton-Brown. 2011. Sequential Model-Based Optimization for General Algorithm Configuration. In *Learning and Intelligent Optimization*. https://api.semanticscholar.org/CorpusID:6944647

[13] Frank Hutter, Lars Kotthoff, and Joaquin Vanschoren. 2019. *Automated Machine Learning - Methods, Systems, Challenges*. doi:10.1007/978-3-030-05318-5

[14] Xiaoyu Jiang, Haotao Xie, Jiayu Wang, Zeyu Yang, Yuanqiang Zhou, Le Yao, and Zheren Zhu. 2026. Agentic AI for Safety-Aware Process Monitoring and Fault Diagnosis: A Review. *Processes* 14, 13 (2026). doi:10.3390/pr14132112

[15] Levente Kocsis and Csaba Szepesvari. 2006. Bandit Based Monte-Carlo Planning. In *European Conference on Machine Learning*. https://api.semanticscholar.org/CorpusID:15184765

[16] Lisha Li, Kevin G. Jamieson, Giulia DeSalvo, Afshin Rostamizadeh, and Ameet Talwalkar. 2016. Hyperband: A Novel Bandit-Based Approach to Hyperparameter Optimization. *J. Mach. Learn. Res.* 18 (2016), 185:1–185:52. https://api.semanticscholar.org/CorpusID:11971778

[17] Liam Li, Kevin G. Jamieson, Afshin Rostamizadeh, Ekaterina Gonina, Jonathan Ben-tzur, Moritz Hardt, Benjamin Recht, and Ameet Talwalkar. 2018. A System for Massively Parallel Hyperparameter Tuning. *arXiv: Learning* (2018). https://api.semanticscholar.org/CorpusID:216245794

[18] Radu Marinescu, Akihiro Kishimoto, Parikshit Ram, Ambrish Rawat, Martin Wistuba, Paulito Palmes, and Adi Botea. 2021. Searching for Machine Learning Pipelines Using a Context-Free Grammar. In *AAAI Conference on Artificial Intelligence*. https://api.semanticscholar.org/CorpusID:235349043

[19] Margaret Mitchell, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. 2018. Model Cards for Model Reporting. *Proceedings of the Conference on Fairness, Accountability, and Transparency* (2018). https://api.semanticscholar.org/CorpusID:52946140

[20] Felix Mohr, Marcel Wever, and Eyke Hüllermeier. 2018. ML-Plan: Automated machine learning via hierarchical planning. *Machine Learning* 107 (2018), 1495–1515. https://api.semanticscholar.org/CorpusID:51886269

[21] Tien-Dung Nguyen, Bogdan Gabrys, and Katarzyna Musial. 2020. AutoWeka4MCPS-AVATAR: Accelerating Automated Machine Learning Pipeline Composition and Optimisation. *Expert Syst. Appl.* 185 (2020), 115643. https://api.semanticscholar.org/CorpusID:227151926

[22] Randal S. Olson and Jason H. Moore. 2019. *TPOT: A Tree-Based Pipeline Optimization Tool for Automating Machine Learning*. Springer International Publishing, Cham, 151–160. doi:10.1007/978-3-030-05318-5_8

[23] Rodrigo Kobashikawa Rosa, Danilo Braga, and Danilo Silva. 2024. Benchmarking deep learning models for bearing fault diagnosis using the CWRU dataset: A multi-label approach. https://api.semanticscholar.org/CorpusID:271328200

[24] Manuel Martin Salvador, Marcin Budka, and Bogdan Gabrys. 2016. Automatic Composition and Optimization of Multicomponent Predictive Systems With an Extended Auto-WEKA. *IEEE Transactions on Automation Science and Engineering* 16 (2016), 946–959. https://api.semanticscholar.org/CorpusID:18001834

[25] Lichen Shi, Jiahang Guo, and Haitao Wang. 2024. A Precision Machining Equipment Fault Diagnosis Based on CWT and Improved ResNeXt. *Instrumentation* 11, 2 (2024), 36–43. doi:10.15878/j.instr.202400030

[26] Jasper Snoek, H. Larochelle, and Ryan P. Adams. 2012. Practical Bayesian Optimization of Machine Learning Algorithms. In *Neural Information Processing Systems*. https://api.semanticscholar.org/CorpusID:632197

[27] Chris J. Thornton, Frank Hutter, Holger H. Hoos, and Kevin Leyton-Brown. 2012. Auto-WEKA: combined selection and hyperparameter optimization of classification algorithms. *Proceedings of the 19th ACM SIGKDD international conference on Knowledge discovery and data mining* (2012). https://api.semanticscholar.org/CorpusID:13952689

[28] João Paulo Vieira, Victor Afonso Bauler, Rodrigo Kobashikawa Rosa, and Danilo Silva. 2025. Towards a more realistic evaluation of machine learning models for bearing fault diagnosis. *ArXiv* abs/2509.22267 (2025). https://api.semanticscholar.org/CorpusID:281659355

[29] Tobias Wagner, Alexander Gepperth, and Elmar Engels. 2023. A framework for the automated parameterization of a sensorless bearing fault detection pipeline. *Journal of Applied Research on Industrial Engineering* 10, 4 (Oct. 2023). doi:10.22105/jarie.2023.391005.1538

[30] Chi Wang, Qingyun Wu, Markus Weimer, and Erkang Zhu. 2019. FLAML: A Fast and Lightweight AutoML Library. In *Conference on Machine Learning and Systems*. https://api.semanticscholar.org/CorpusID:229348714

[31] Haitao Wang and Xiang Liu. 2024. Research on Rotating Machinery Fault Diagnosis Based on Improved Multi-target Domain Adversarial Network. *Instrumentation* 11, 1 (2024), 38–50. doi:10.15878/j.instr.202300151

[32] Chen Yang, Jianwen Yan, Yixiong Feng, Lei Li, and Jianrong Tan. 2026. Hybrid Deep Learning for Hydraulic Cylinder Fault Diagnosis under Complex Conditions via Multi-Source Signal Fusion. *Instrumentation* 13, 1 (2026), 40–56. doi:10.15878/j.instr.202600318

[33] Marc-André Zöller and Marco F. Huber. 2019. Benchmark and Survey of Automated Machine Learning Frameworks. *J. Artif. Intell. Res.* 70 (2019), 409–472. https://api.semanticscholar.org/CorpusID:210064426